

检查点 5 实验报告

手写训练循环、训练监控与学习现象可视化

2026 年 4 月 18 日

目录

1 实验目标	3
2 工程结构与本阶段新增内容	3
3 模型、标签与损失设置	4
3.1 模型输入输出与张量形状	4
3.2 标签编码方式	4
3.3 当前损失函数	5
4 训练配置与手写训练循环	6
4.1 TOML 配置文件	6
4.2 实际训练规模	6
4.3 手写训练循环	7
5 监控指标设计	7
5.1 记录的主要指标	7
5.2 为什么选择这些指标	8
6 训练结果与曲线分析	8
6.1 epoch 级结果汇总	8
6.2 loss 曲线	9
6.3 loss 曲线解释	10
6.4 学习率与梯度范数	10
6.5 验证指标与补充曲线	11
6.6 为什么当前检测效果仍然很差	13
7 训练过程预测框演化	14
7.1 可视化方式	14
7.2 训练过程快照示例	14
7.3 可视化现象解释	15

8 良好习惯与工程规范	15
9 结论	16
A 附录：checkpoint 5 相关脚本与工具说明	16

1 实验目标

本次检查点 5 在检查点 4 已经跑通极简检测链路的基础上，继续向一个更完整的训练工程推进。核心目标包括：

1. 不依赖 Trainer 类，手写完整的训练循环。
2. 在 TensorBoard 中接入训练损失、学习率、梯度范数、验证集 mAP@0.5 等指标。
3. 固定验证样本，观察训练过程中预测框是否逐步向真实框靠拢。
4. 将训练超参数从代码中抽离到 TOML 配置文件，并在训练结束后保存训练结果文本与配置快照。

2 工程结构与本阶段新增内容

本阶段继续在 /home/lidz/YOLO/miniyolo 下进行开发，其中与 checkpoint 5 直接相关的核心模块如下：

- configs/train_base.toml：训练配置文件。
- tools/train_checkpoint5.py：手写训练循环主脚本。
- utils/simple_toml.py：最小 TOML 读取工具。
- utils/metrics.py：验证阶段 mAP@0.5 与辅助指标计算。
- utils/prediction.py：预测框解码与 IoU 计算。
- utils/visualization.py：固定样本训练过程快照可视化。
- tools/export_checkpoint5_curves.py：将 TensorBoard 事件文件导出为报告用图片。

本阶段最终产出的核心结果包括：

- TensorBoard 事件文件：runs/checkpoint5_fulltrain/
- 训练结果文本：logs/train_results/checkpoint5_fulltrain_20260418_175839.txt
- 配置快照：logs/train_results/checkpoint5_fulltrain_20260418_175839.toml
- 模型权重：outputs/checkpoint5/checkpoint5_fulltrain_20260418_175839.pth
- 训练过程可视化：outputs/checkpoint5/checkpoint5_fulltrain_progress/
- 曲线图片：outputs/checkpoint5/curves/

3 模型、标签与损失设置

3.1 模型输入输出与张量形状

本阶段继续使用 checkpoint 4 中已经跑通的极简 MiniYOLO 结构：

- 输入图像：[B, 3, 224, 224]
- backbone 输出特征：[B, 512, 7, 7]
- detection head 输出预测：[B, 7, 7, 84]

这里输出维度之所以是 84，是因为当前训练采用混合的 VOC+COCO 统一数据入口，类别数设置为 80，每个格子额外预测一个框参数向量 (x, y, w, h)，因此：

$$80 + 4 = 84$$

进一步将输出拆分后，可得到：

- 分类预测：pred_cls = [B, 7, 7, 80]
- 边界框预测：pred_box = [B, 7, 7, 4]

3.2 标签编码方式

真实标注框在统一后的 manifest 中采用 xyxy 格式，即：

$$[x_1, y_1, x_2, y_2]$$

训练前先将其转换为：

$$c_x = \frac{x_1 + x_2}{2}, \quad c_y = \frac{y_1 + y_2}{2}, \quad w = x_2 - x_1, \quad h = y_2 - y_1$$

随后将中心点与宽高按图像尺寸 224 做归一化，并映射到 7x7 网格：

$$grid_x = \lfloor c_x \times 7 \rfloor, \quad grid_y = \lfloor c_y \times 7 \rfloor$$

对应的格子内部偏移为：

$$cell_x = c_x \times 7 - grid_x, \quad cell_y = c_y \times 7 - grid_y$$

因此，每张图的监督被编码为 3 个张量：

- target_cls: [7, 7, 80]
- target_box: [7, 7, 4]
- object_mask: [7, 7]

其中:

- `target_cls` 表示 one-hot 分类监督。
- `target_box` 保存 (`cell_x`, `cell_y`, `w`, `h`)。
- `object_mask` 标记当前格子是否负责一个真实目标。

3.3 当前损失函数

本实验仍采用极简 YOLO 风格损失, 仅保留:

1. 分类损失 `cls_loss`
2. 边界框回归损失 `box_loss`

不包含:

- objectness / confidence 分支
- anchor
- 多尺度预测

具体计算方式为:

$$L_{cls} = \frac{\sum((pred_cls - target_cls)^2 \cdot mask)}{N_{pos} \times 80}$$
$$L_{box} = \frac{\sum((pred_box - target_box)^2 \cdot mask)}{N_{pos} \times 4}$$

总损失为:

$$L = \lambda_{cls} \cdot L_{cls} + \lambda_{box} \cdot L_{box}$$

当前实验配置中:

$$\lambda_{cls} = 1.0, \quad \lambda_{box} = 5.0$$

因此:

$$L = L_{cls} + 5 \cdot L_{box}$$

这种设置意味着当前训练更加重视框回归误差, 因此总损失的下降趋势主要受到 `box_loss` 影响。

4 训练配置与手写训练循环

4.1 TOML 配置文件

本阶段将训练超参数完全移出代码，统一放在 `configs/train_base.toml` 中。当前主训练配置如表 1 所示。

表 1: checkpoint 5 主训练配置

配置项	当前设置
训练集 manifest	<code>all_train.jsonl</code>
验证集 manifest	<code>all_val.jsonl</code>
输入尺寸	224
网格大小	7
类别数	80
训练样本上限	200000
验证样本上限	200000
batch size	256
epochs	20
学习率	$1e-3$
最小学习率	$1e-5$
学习率调度器	cosine
DataLoader worker 数	64
pin memory	True
persistent workers	True
prefetch factor	4
多卡并行	DataParallel (8 x RTX 4090)
验证间隔	每 5 个 epoch
可视化间隔	每 5 个 epoch
mAP 阈值	$\text{IoU} = 0.5$
top-k 预测框	3
score threshold	0.0

4.2 实际训练规模

尽管配置中训练集和验证集上限都写成了 200000，但本轮实际可用样本规模由 manifest 本身决定，因此实际训练规模为：

- 训练集：124004 张图像
- 验证集：10823 张图像

在 `batch_size=256` 的条件下，每个 epoch 大约包含：

$$\frac{124004}{256} \approx 485$$

个训练 step，20 个 epoch 合计约 9700 个训练 step。

4.3 手写训练循环

本阶段训练循环完全手写实现，没有借用高层 Trainer 类，主流程可概括为：

1. 从 DetectionDataset 和 DataLoader 读取 batch
2. 执行模型前向传播
3. 计算 total_loss、cls_loss、box_loss
4. 执行 loss.backward()
5. 执行 optimizer.step()
6. 执行 optimizer.zero_grad()

因此 checkpoint 5.1 要求中的：

```
for epoch in range(n): for batch in dataloader:
```

式训练结构已经完全落实到了训练脚本中。

5 监控指标设计

5.1 记录的主要指标

本阶段在 TensorBoard 中记录了以下几类指标：

1. 损失类指标

- loss/total_step
- loss/classification_step
- loss/box_step
- loss/total_epoch
- loss/classification_epoch
- loss/box_epoch

2. 优化过程指标

- train/lr
- train/lr_epoch

- grad_norm/global_total
- 各层参数的 grad_norm/...

3. 验证指标

- val/map50
- val/mean_pred_score
- val/num_predictions
- val/num_present_classes

4. 训练数据统计

- train/positive_cells_per_image

5.2 为什么选择这些指标

这些指标分别回答不同问题：

- loss: 模型是否在学习。
- lr: 学习率调度是否正常执行。
- grad_norm: 梯度是否过大、过小或爆炸。
- mAP@0.5: 验证集检测性能是否提升。
- mean_pred_score / num_predictions: 模型预测行为如何变化。
- positive_cells_per_image: 每张图平均有多少正样本格子，帮助解释 loss 波动。

6 训练结果与曲线分析

6.1 epoch 级结果汇总

根据本轮训练生成的结果文本，可提炼出关键 epoch 级结果如表 2 所示。

表 2: checkpoint 5 关键 epoch 结果

Epoch	Total Loss	Cls Loss	Box Loss	mAP@0.5
01	0.3546	0.0130	0.0683	—
05	0.2581	0.0123	0.0492	0.0001
10	0.2275	0.0119	0.0431	0.0002
15	0.2068	0.0117	0.0390	0.0003
20	0.1944	0.0117	0.0365	0.0003

从结果可以直接观察到：

- 总损失从 0.3546 下降到 0.1944
- 分类损失从 0.0130 下降到 0.0117
- 边界框损失从 0.0683 下降到 0.0365
- mAP@0.5 从 0.0001 轻微上升到 0.0003

6.2 loss 曲线

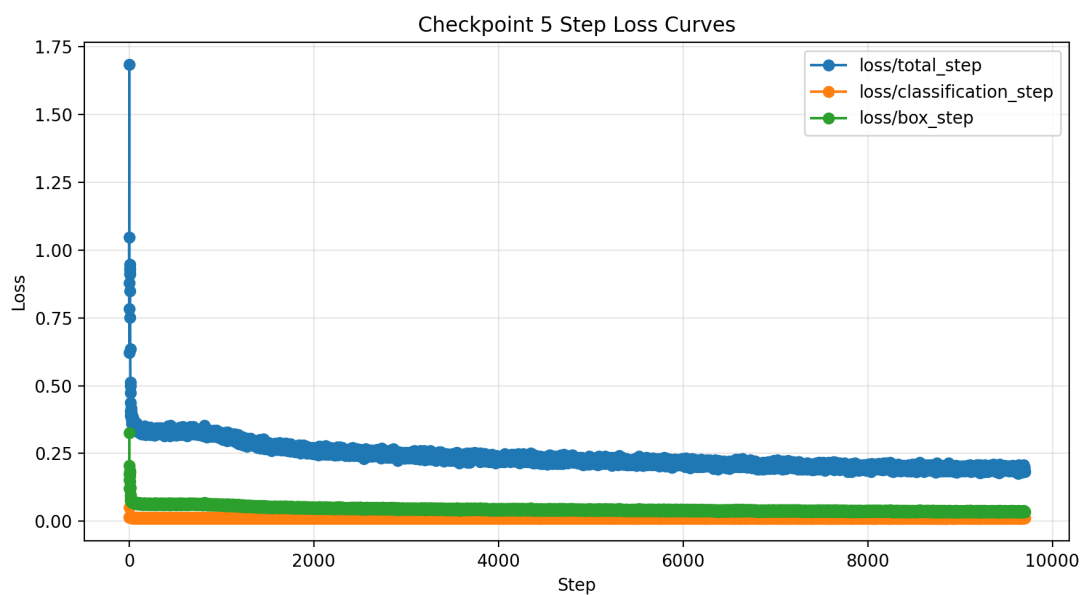


图 1: step 级别损失曲线

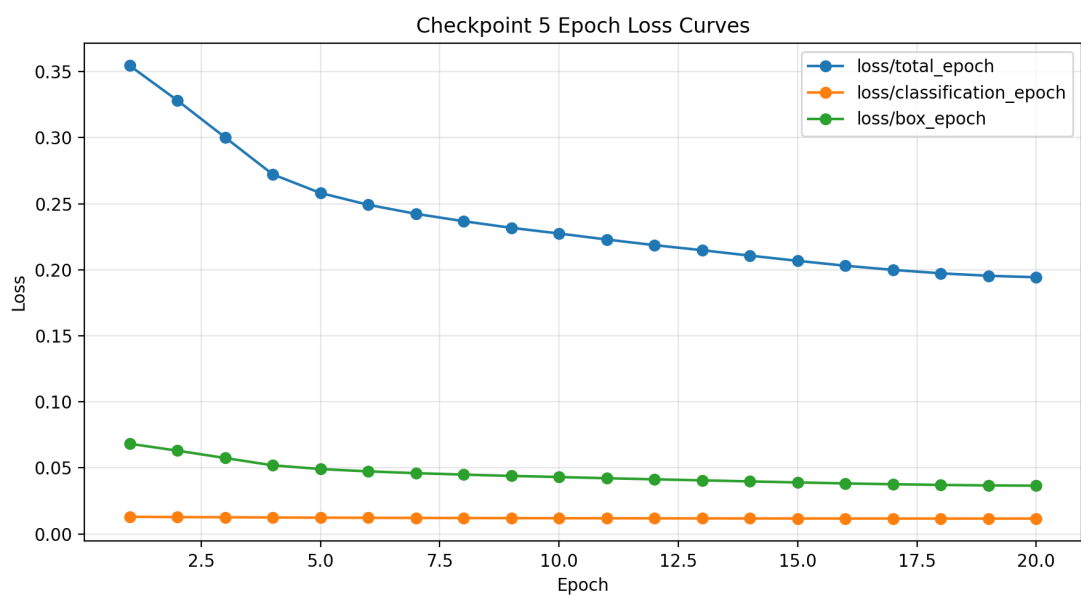


图 2: epoch 级别损失曲线

6.3 loss 曲线解释

- **step 曲线**直接记录每个 batch 的原始 loss 值，因此波动较大。
- **epoch 曲线**是一个 epoch 内所有 batch 的平均值，因此更平滑，也更适合总结收敛趋势。
- 当前总损失持续下降，说明模型确实在学习当前监督目标。
- **box_loss** 的下降更明显，这是因为总损失里对框回归给予了更大的权重。

6.4 学习率与梯度范数

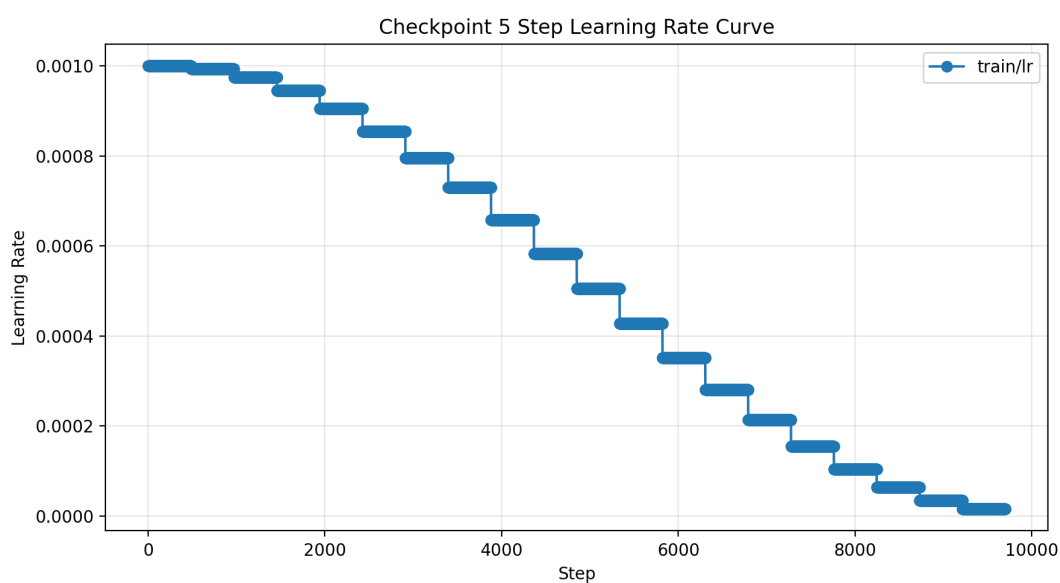


图 3: step 级别学习率曲线

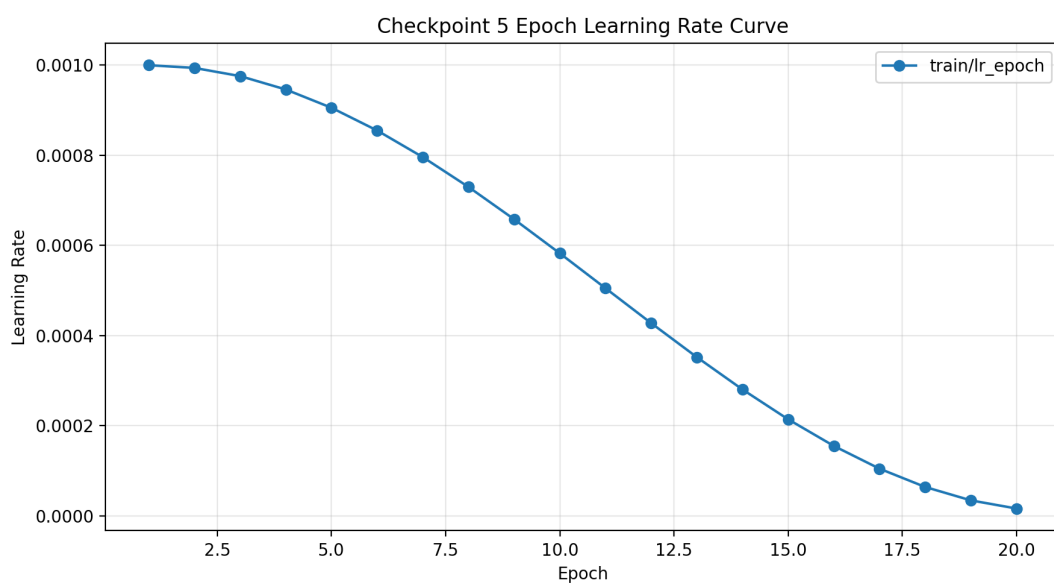


图 4: epoch 级别学习率曲线

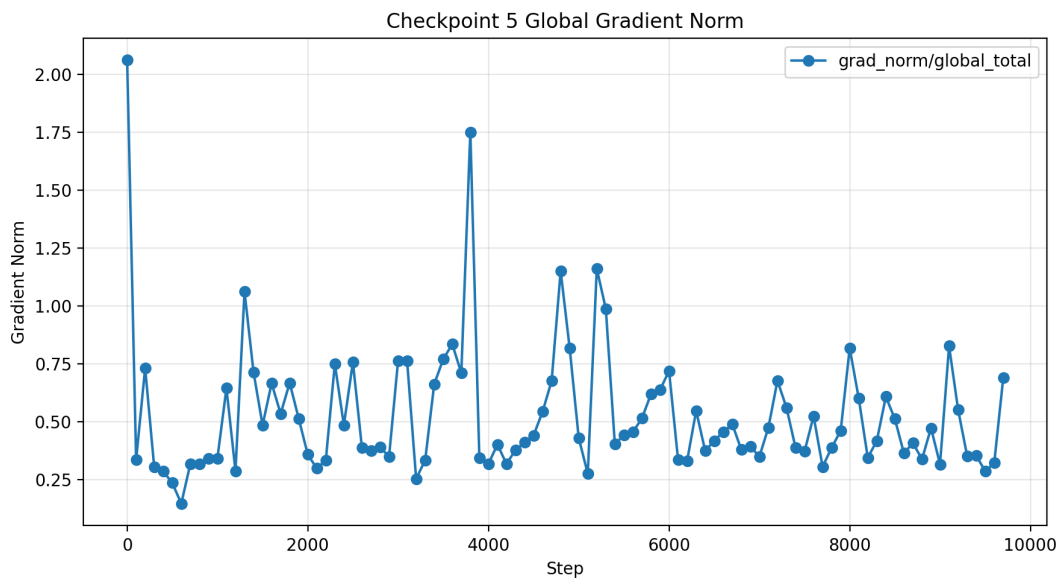


图 5: 全局梯度范数曲线

从这些曲线可以看出：

- 学习率按照余弦退火策略，从 $1e-3$ 逐步下降到约 $1.61e-5$ 。
- 从当前全局梯度范数曲线看，最终约为 0.69，未观察到明显梯度爆炸。
- 在现有监控范围内，训练过程表面上数值稳定。

6.5 验证指标与补充曲线

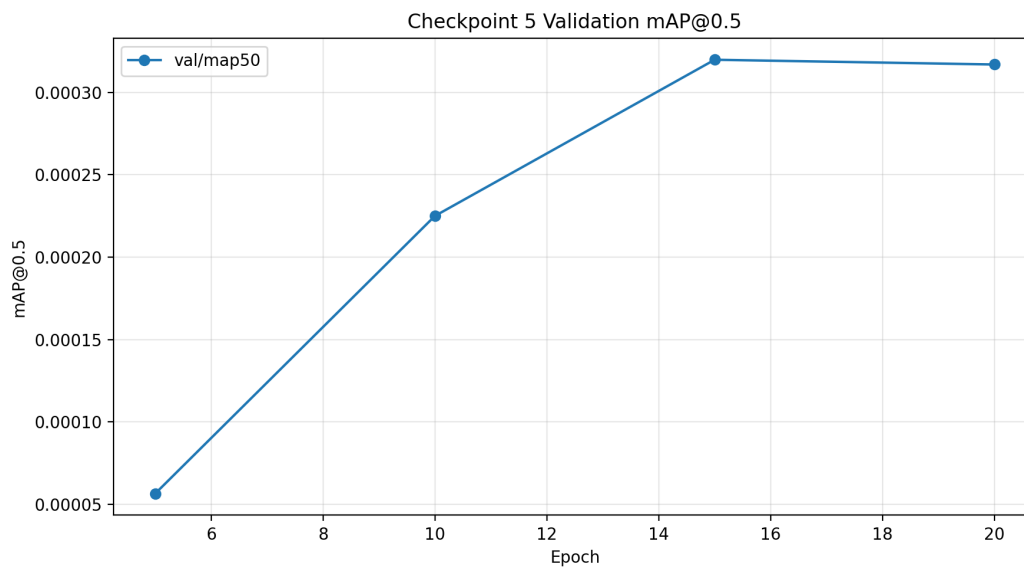


图 6: 验证集 mAP@0.5 曲线

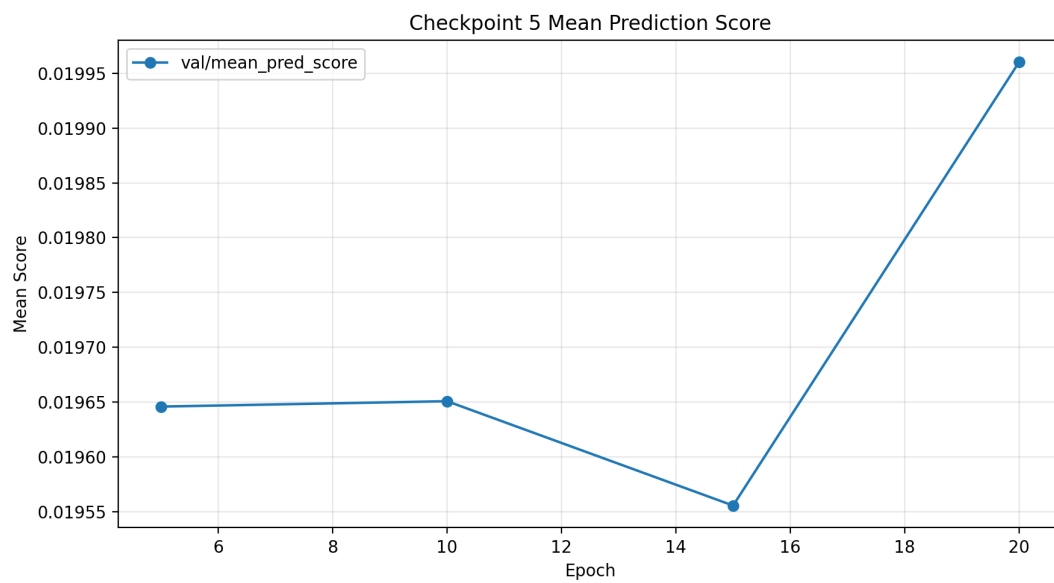


图 7: 平均预测分数曲线

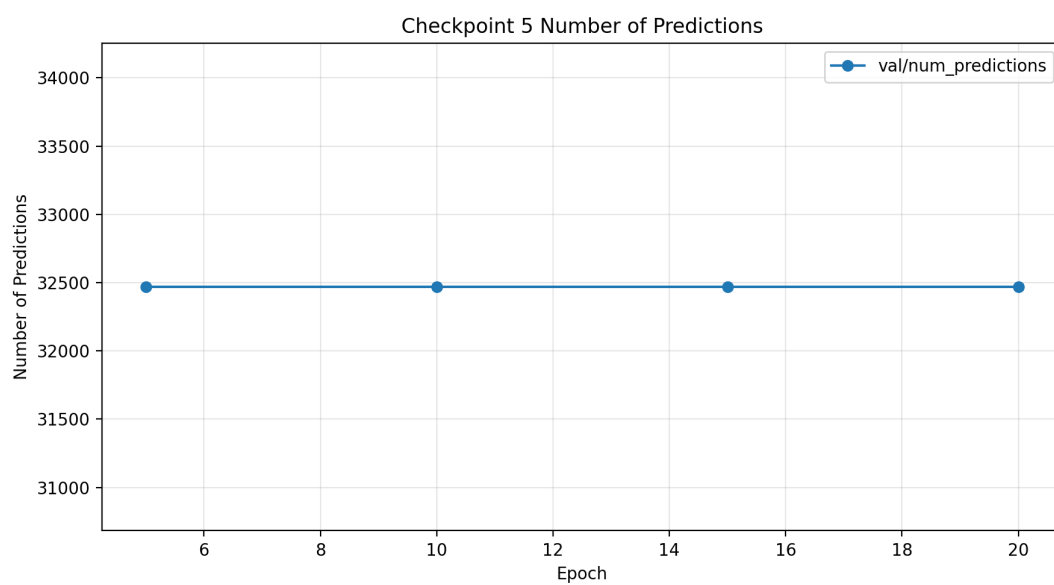


图 8: 预测框数量曲线

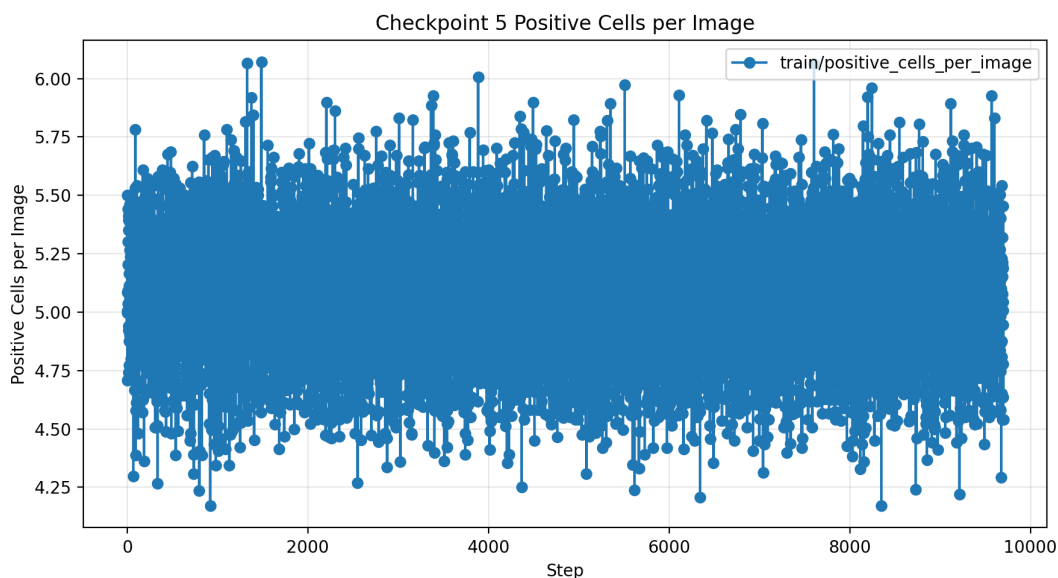


图 9: 每张图平均正样本格子数曲线

对验证指标的解释如下：

- **mAP@0.5** 虽然有小幅提升，但整体仍非常低，说明检测性能很弱。
- **mean_pred_score** 最终约为 0.02，说明模型输出分数整体偏低。
- **num_predictions** 在本轮验证中约为 32469，这是因为当前验证设置固定每张图最多保留 3 个预测框，而验证集共有约 10823 张图像。
- **positive_cells_per_image** 最终约为 4.54，说明平均每张图有多个正样本网格，监督信号并不稀疏。

6.6 为什么当前检测效果仍然很差

尽管 loss 持续下降，但当前 **mAP@0.5** 依旧很低，主要原因如下：

1. 当前极简模型没有 **objectness** / **confidence** 分支，无法有效区分背景格子 and 真正含目标的格子。
2. 分类损失只在正样本格子上计算，背景格子没有直接分类惩罚，容易产生大量伪预测。
3. 推理阶段固定采用 **top-k=3** 且 **score_threshold=0.0**，会保留较多低质量预测框，从而显著拉低精度。
4. 模型本身非常轻量，输入分辨率仅 224，输出网格只有 7x7，定位粒度较粗。
5. 当前损失函数过于简化，只使用了分类 MSE 和框回归 MSE，没有引入更成熟的检测训练机制。
6. 当前训练数据混合了 VOC 与 COCO，共 80 类，对于极简模型而言任务本身就较难。

因此，本轮训练更应该被看作：

在真实混合数据上成功跑通了手写训练、监控和预测演化观察的完整链路，而不是已经获得高质量检测器。

7 训练过程预测框演化

7.1 可视化方式

为了满足 checkpoint 5.3 的要求，本实验固定了 4 张验证图像，并在：

- epoch 000
- epoch 005
- epoch 010
- epoch 015
- epoch 020

分别保存预测快照图。图中：

- 绿色框表示真实框（GT）
- 红色框表示预测框（Pred）

7.2 训练过程快照示例

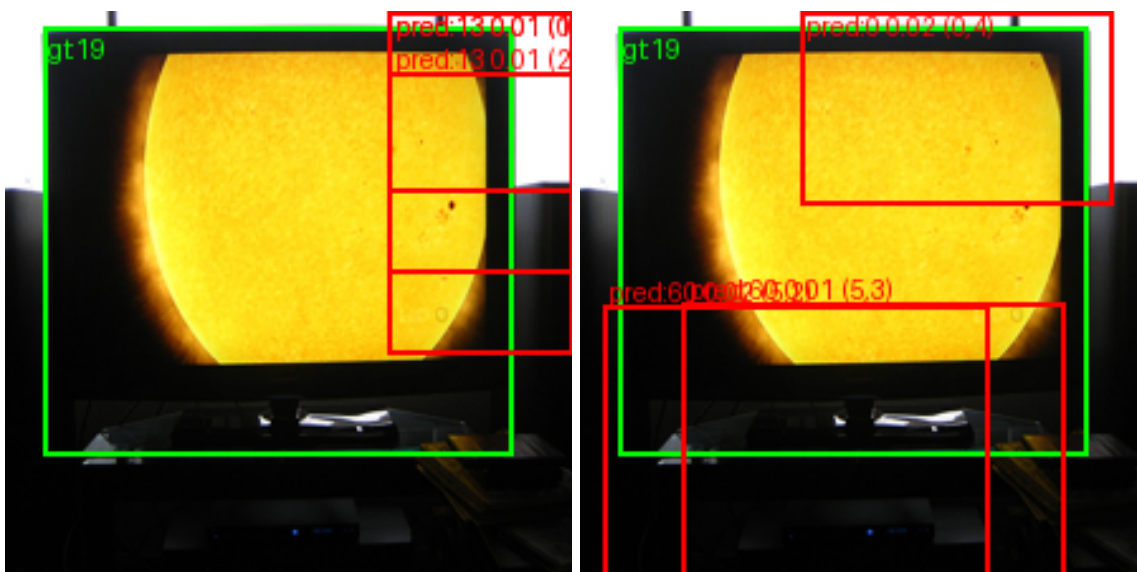


图 10: 固定样本 voc2012_val_2008_000002 在 epoch 000 与 epoch 020 的预测对比



图 11: 固定样本 voc2012_val_2008_000003 在 epoch 000 与 epoch 020 的预测对比

7.3 可视化现象解释

从这些快照中可以观察到：

- 在 epoch 000 时，预测框更接近随机输出，和真实框位置关联较弱。
- 随着训练进行，部分预测框的位置与目标区域开始出现一定相关性。
- 但由于当前模型极简、缺少 objectness，而且推理时阈值较低，红色预测框整体仍较混乱。

因此，本阶段的结论不是“模型已经检测得很好”，而是：

训练过程中的预测框确实发生了变化，说明模型输出并非固定随机噪声，而是在逐步受真实监督影响。

8 良好习惯与工程规范

本阶段还落实了 checkpoint 5.4 中的工程规范：

1. **超参数不再硬编码**：所有核心训练参数统一写入 TOML 文件。
2. **每次训练保留结果文本**：训练结束后保存 .txt 结果摘要。
3. **每次训练保留配置快照**：将本次实际使用的 TOML 保存为快照，便于复现实验。
4. **每次训练保留模型权重与可视化结果**：所有关键产物统一收敛到 outputs/checkpoint5/。

本轮训练对应的结果文本与配置快照分别为：

- logs/train_results/checkpoint5_fulltrain_20260418_175839.txt
- logs/train_results/checkpoint5_fulltrain_20260418_175839.toml

9 结论

本次 checkpoint 5 已经在真实混合数据上完成了一个更完整的目标检测训练工程，主要结论如下：

1. 已经成功实现了完全手写的训练循环，并在 8 张 RTX 4090 上以 DataParallel 方式完成了真实数据训练。
2. 已经将损失、学习率、梯度范数、mAP@0.5 以及多项辅助指标接入 TensorBoard。
3. 已经完成固定样本的训练过程快照保存，能够展示预测框从初始随机状态到训练后状态的变化过程。
4. 训练损失显著下降，说明模型确实在学习当前监督目标。
5. 当前 mAP@0.5 提升很有限，说明极简模型和极简损失虽然能跑通链路，但检测效果仍然较弱。
6. 配置驱动训练、结果文本保存和配置快照保存等工程规范已经落实，为后续更强模型和更长训练提供了基础。

A 附录：checkpoint 5 相关脚本与工具说明

脚本/文件	作用
configs/train_base.toml	checkpoint 5 主训练配置文件，统一管理数据路径、batch size、epochs、学习率、验证与可视化间隔等超参数。
utils/simple_toml.py	读取项目所需的简单 TOML 配置，避免训练脚本中硬编码超参数。
models/minimal_backbone.py	定义极简 CNN backbone，将输入图像压缩为 7x7 特征图。
models/detection_head.py	将 backbone 输出映射为每个网格位置的分类与 bbox 预测。
models/miniyolo.py	将 backbone 与 detection head 组合为完整 MiniYOLO 模型。
data/target_encoder.py	将真实框与类别编码为 target_cls、target_box 和 object_mask。
data/detection_dataset.py	从 Unified manifest 读取真实样本，完成图片 resize、框缩放和监督编码。
losses/minimal_yolo_loss.py	定义当前使用的极简 YOLO 风格损失函数。
utils/prediction.py	负责预测框解码、分类分数处理以及 IoU 计算。

脚本/文件	作用
<code>utils/metrics.py</code>	负责验证阶段 mAP@0.5 及相关辅助指标计算。
<code>utils/visualization.py</code>	统一 GT 与 Pred 的绘图逻辑，并保存训练过程固定样本快照。
<code>tools/inspect_model.py</code>	检查模型每层 tensor shape，验证 backbone 与 head 的输出尺寸。
<code>tools/test_target_encoder.py</code>	验证目标编码是否正确写入 7x7 网格监督。
<code>tools/test_minimal_yolo_loss.py</code>	验证极简 YOLO Loss 是否能输出标量并反向传播。
<code>tools/test_detection_dataset.py</code>	验证真实数据版 Dataset 与 DataLoader 是否正常工作。
<code>tools/train_checkpoint5.py</code>	checkpoint 5 主训练脚本，负责手写训练循环、TensorBoard 记录、验证、可视化与结果保存。
<code>tools/visualize_predictions.py</code>	单独加载训练后模型，在真实样本上绘制预测框与真实框对比图。
<code>tools/export_checkpoint5_curves.py</code>	从 TensorBoard 事件文件导出 checkpoint 5 报告所需曲线图片。
TensorBoard	训练监控工具，用于查看 loss、学习率、梯度范数和验证指标曲线。
<code>runs/checkpoint5_fulltrain/</code>	保存本轮训练产生的 TensorBoard 事件文件。
<code>outputs/checkpoint5/</code>	保存模型权重、训练过程快照与导出的曲线图。
<code>logs/train_results/</code>	保存每次训练的结果文本和配置快照。
<code>logs/checkpoint5_materials.md</code>	汇总 checkpoint 5 报告可直接引用的材料清单。